МИНОБРНАУКИ РОССИИ

РТУ МИРЭА

Контрольная работа № 2

по дисциплине «Создание программного обеспечения»

Документация backend, frontend и API

Продолжение User Story Map продукта FrameSpace

Группа ЭФБО-01-24

Москва, 2026

1. Что именно продолжается

Backbone карты, слева направо:

1. Найти площадку.

2. Оформить заявку.

3. Согласовать.

4. Оплатить.

Горизонтальные полосы карты — вехи. В эту работу входит только верхняя полоса, Release 1 «Цифровой приём». Нижние полосы не проектируются заново и не помечаются сделанными.

Колонка карты	Карточка Release 1	Что делаем в этой работе
Найти площадку	Каталог локаций студии: фото, адрес, вместимость	Каталог по уже существующим полям локации
Оформить заявку	Веб-форма: название, дата, бюджет	Форма поверх правил заявки
Согласовать	Администратор меняет статус в веб-панели	Панель и переход статуса по графу
Оплатить	Прочерк	Нет ни экрана, ни метода, ни статуса оплаты

Прочерк в колонке «Оплатить» сохраняется. Онлайн-оплата — карточка Release 3, не часть приёма заявки.

Результат этапа из дорожной карты, к которому относится документ: клиент создаёт заявку и видит её номер и статус без звонка, администратор обрабатывает заявки в веб-панели, а не в консоли. Календарь занятости, письмо клиенту, маркетплейс и оплата остаются следующими полосами карты.

Доменное ядро контрольной работы № 1 не подменяется. Клиент, локация и заявка, граф статусов и правила сервисов остаются источником правды. Консоль **Main** не удаляется.

2. Матрица: карточка карты → три слоя

Карточка Release 1	Backend	Frontend	API
Каталог локаций	Таблица **locations**, **LocationService.getAll**	Экран «Каталог»	**GET /api/locations**
Веб-форма заявки	**ShootingRequestService.create**, статус всегда **NEW**	Экран «Новая заявка»	**POST /api/shooting-requests**
Номер созданной заявки	Поле **id**, которое возвращает база	Экран подтверждения	Тело ответа **201**, поле **id**
Ошибка, если дата в прошлом	Исключение сервиса до записи	Сообщение у поля даты, запрос не считается успешным	**400**, тело **{ "error": "..." }**
Смена статуса администратором	**ShootingRequestService.changeStatus**	Экран «Заявки», действие «Сменить статус»	**PATCH /api/shooting-requests/{id}/status**
Оплата	Нет	Нет	Нет

Фотография площадки стоит на карточке каталога, но в проверенной модели локации её нет: есть название, адрес, вместимость и тип. В Release 1 каталог показывает эти поля. Отдельное хранение снимка не добавляется, иначе первая полоса карты раздуется за счёт ещё не проверенной сущности. Это сознательный срез объёма, а не молчаливый перенос вехи.

Письмо с номером заявки входит в критерии приёмки Story 1 и в Definition of Done той истории. В этом инкременте события почты нет. История из-за этого не считается закрытой целиком. Почтовая

отправка остаётся отдельной карточкой внутри колонки «Оформить заявку», ниже формы, и не блокирует приём заявки через API.

3. Backend

Слои те же, что в прототипе:

```
Экран или консоль → сервис → репозиторий JDBC → PostgreSQL
```

SQL остаётся в репозиториях. Параметры передаются через **PreparedStatement**. Правила пустого названия, даты в прошлом, чужого клиента, чужой локации, нулевой длительности, отрицательного бюджета и запрещённого перехода статуса остаются в сервисах. API их не копирует, а вызывает.

3.1. Таблицы, которые карта уже подтвердила

clients — заказчик съёмки: ФИО, уникальная почта, телефон. Один клиент — много заявок. Удаление клиента удаляет его заявки (**ON DELETE CASCADE**). Заказчик в систему как сотрудник не входит, пароля у него нет.

locations — площадка: уникальное название, адрес, вместимость больше нуля, тип **PAVILION**, **OUTDOOR**, **STUDIO** или **OFFICE**. Удаление площадки, на которую ссылаются заявки, база запрещает (**ON DELETE RESTRICT**). Сервис дополнительно не даёт удалить локацию, пока на неё есть заявки в статусах **NEW**, **APPROVED** или **IN_PROGRESS**.

shooting_requests — заявка: **client_id**, **location_id**, название, описание, дата, длительность больше нуля, бюджет не меньше нуля, статус. Новая заявка создаётся только в **NEW**.

Граф, уже зафиксированный в **ShootingRequestService**:

- **NEW** → **APPROVED** или **REJECTED**
- **APPROVED** → **IN_PROGRESS** или **CANCELLED**
- **IN_PROGRESS** → **COMPLETED** или **CANCELLED**
- из **COMPLETED**, **REJECTED** и **CANCELLED** переходов нет

Файл **sql/schema.sql** этими таблицами ограничен и в этой работе не расширяется.

3.2. Таблица входа, которой на карте ещё не было

Чтобы панель администратора не смешивалась с заказчиком, добавляется **users**: логин, bcrypt-хеш пароля, ФИО, время создания. Внешнего ключа на **clients** нет. Определение лежит в **sql/users.sql**. При старте API та же таблица создаётся командой **CREATE TABLE IF NOT EXISTS**, если скрипт ещё не применяли.

Демонстрационный сотрудник, если логина ещё нет: **admin** / **admin123**. Это строка **users**, не клиент киностудии.

3.3. Чего в базе Release 1 нет специально

Нет календаря слотов и проверки пересечения дат. Это карточка Release 2 «Выбор свободной даты». Сейчас две заявки на одну локацию и одну дату допустимы: карта относит защиту от двойного бронирования ко второй вехе.

Нет статуса «оплачено», платежа и комиссии. Колонка «Оплатить» в Release 1 пустая.

Нет таблицы студий. Мультистудийный каталог — Release 3.

4. Frontend

Веб-клиент Release 1 — это три экрана по трём заполненным карточкам карты плюс вход в панель. Отдельного интерфейсного приложения в ветке нет: ниже зафиксирован состав экранов, чтобы карта имела прямое экранное продолжение, а реализация панели шла поверх уже существующего API.

Общие правила экранов:

- данные приходят только из API, свои правила заявок экран не хранит;
- текст ошибки показывается из поля **error**;
- дата вводится календарём или полем **гггг-мм-дд** и уходит на сервер в этом виде;
- после каждого успешного действия виден результат: список, номер заявки или новый статус.

4.1. Вход в панель

Экран для администратора студии, колонка «Согласовать».

Поля регистрации: логин, пароль, ФИО. Поля входа: логин и пароль. Логин — 3–50 символов, латиница, цифры и подчёркивание. Пароль не короче 6 символов. После входа сохраняется токен и заголовок **Authorization: Bearer**. Чужой логин и неверный пароль показывают одну и ту же фразу, без подсказки, существует ли учётная запись.

4.2. Каталог. Колонка «Найти площадку»

Список карточек локации. На карточке: название, адрес, вместимость, тип. Снимка нет, пока его нет в модели. Выбор карточки открывает форму заявки с уже подставленной локацией. Пустой каталог — отдельное состояние «Площадок пока нет», не ошибка сети.

4.3. Форма заявки. Колонка «Оформить заявку»

Поля, обязательные по Story 1 и по сервису: название, описание, дата, длительность в часах, бюджет, выбранная локация, заказчик. Статус на форме не выбирается.

Кнопка «Отправить»:

- успешный ответ показывает номер заявки и статус **NEW**;
- дата в прошлом подсвечивает поле и оставляет человека на форме;
- пустое название, пустое описание, нулевая длительность и отрицательный бюджет тоже остаются на форме с текстом сервиса;
- сеть и ответ **500** не создают вторую заявку сами по себе: повтор нажимает человек.

Отдельной галочки согласия на обработку персональных данных и отправки письма в этом экране нет. Оба пункта входят в Definition of Done Story 1 и остаются открытыми, пока не выполнены. Форма из-за них не объявляется законченной историей.

4.4. Панель заявок. Колонка «Согласовать»

Таблица заявок: номер, название, дата, локация, бюджет, статус. Фильтр один за раз, в том порядке, в каком его принимает API: фрагмент названия, локация, статус, диапазон дат, сортировка по дате или бюджету.

Действие «Сменить статус» предлагает только разрешённые следующие статусы. Если сервер отклонил переход, в строке остаётся прежний статус и показывается текст ошибки. Удаление локации с активными заявками из каталога администратора показывает отказ сервиса и не убирает карточку.

Колонки «Оплатить» на панели нет.

5. API

Базовый адрес: **http://localhost:7070**. Тела: **application/json**.

Без токена доступны только:

- **GET /api/health**
- **POST /api/auth/register**
- **POST /api/auth/login**

Остальные маршруты требуют **Authorization: Bearer <token>**. Токен живёт один час, подпись HMAC-SHA256, секрет не короче 32 символов и читается из **JWT_SECRET**. Пароль в токен не входит.

Успешные регистрация и вход:

```
{
  "token": "<jwt>",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "id": 1,
  "username": "admin",
  "fullName": "Администратор"
}
```

5.1. Методы по колонкам карты

Метод и путь	Колонка карты	Действие
GET /api/locations	Найти площадку	Список каталога
POST /api/locations	Служебное ведение каталога	Создать площадку
GET, **PUT**, **DELETE /api/locations/{id}**	Служебное ведение каталога	Карточка, правка, удаление
POST /api/shooting-requests	Оформить заявку	Создать заявку в статусе **NEW**
GET /api/shooting-requests	Согласовать	Список и один фильтр
GET, **PUT**, **DELETE /api/shooting-requests/{id}**	Согласовать	Карточка, правка, удаление
PATCH /api/shooting-requests/{id}/status	Согласовать	Смена статуса
GET, **POST /api/clients** и **GET**, **PUT**, **DELETE /api/clients/{id}**	Оформить заявку	Заказчик, на которого ссылается заявка
GET /api/statistics	Панель студии, не отдельная колонка карты	Те же показатели, что пункт «Статистика» консоли

Тело заявки: **clientId**, **locationId**, **title**, **description**, **shootingDate** (**гггг-мм-дд**), **durationHours**, **budget**.

Смена статуса: **{ "status": "APPROVED" }**.

Список заявок принимает одно условие. Если заданы несколько, берётся первое по порядку: **title**, **location**, **status**, пара **from** и **to**, **sort** со значением **date** или **budget**.

5.2. Коды, которые видит экран

Код	Когда	Что делает экран
200	Чтение или изменение	Показывает данные
201	Созданы заявка, клиент, локация или учётка	Показывает номер или выполняет вход
204	Удаление	Убирает строку без тела

Код	Когда	Что делает экран
400	Пустое поле, дата в прошлом, недопустимый переход	Текст у формы, данные на экране не затираются
401	Нет или испорчен токен	Возврат на экран входа
404	Нет записи	«Не найдено», список не ломается
409	Заняты почта, название локации или логин	Текст у соответствующего поля
500	Прочая ошибка SQL	Общее сообщение, без повторной отправки само по себе

Тело ошибки: **{ "error": "текст" }**.

Создание заявки вызывает сервис буквально так: клиент, локация, название, описание, дата, длительность, бюджет. Поэтому пустое название, несуществующий заказчик, дата в прошлом и переход из **NEW** сразу в **COMPLETED** отклоняются там же, где и в консоли.

6. Story 1 на трёх слоях

История карты: как клиент студии, я хочу заполнить заявку через веб-форму, чтобы не звонить для первичной фиксации.

Критерий приёмки	Backend	Frontend	API
Обязательные поля и кнопка «Отправить» сохраняют заявку в **NEW**, виден номер	**create** пишет статус **NEW** и возвращает сущность с **id**	Экран подтверждения показывает номер	**POST /api/shooting-requests → 201**
Дата в прошлом не отправляет заявку	Сервис бросает исключение до **INSERT**	Поле даты остаётся заполненным, показывается текст	**400**
Письмо с номером и статусом	Не делается	Не делается	Нет метода

Первые два критерия закрываются текущими сервисом и API. Третий не закрывается. По правилу, уже принятому в разборе сбоев на карте, история не помечается Done, пока письмо не выделено и не сдано отдельно.

7. Что остаётся на следующих полосах карты

Release 2. Календарь занятости, фильтр по вместимости и типу, выбор только свободной даты, письмо при смене статуса. Пока фильтр типа и вместимости на экране каталога можно включить локально по уже пришедшему списку, но запрет занятой даты появится только вместе с правилом сервиса. Иначе экран пообещает защиту, которой база не даёт.

Release 3. Каталог нескольких студий, маршрутизация заявки, онлайн-оплата. Колонка «Оплатить» заполняется здесь, не в Release 1.

Release 4+. Рекомендации, шаблоны повторных заявок, бюджет в реальном времени, предоплата. В эту документацию не входят.

8. Где лежит реализация этого среза

Ветка **spo/kr2-api-jwt-cicd** репозитория **framespace_kinostudio**.

- Backend домена: прежние сервисы и **sql/schema.sql**.
- Вход сотрудника: **sql/users.sql**, пакет **ru.mirea.kinostudio.api.auth**.
- API: **ru.mirea.kinostudio.api**, контракт также в **docs/api-contract.md**.
- Проверка без базы: **mvn -B verify** (Checkstyle пакета **api** и модульные тесты входа и статуса).
- Frontend Release 1 описан в этом файле и ждёт отдельного веб-клиента к тем же методам.

Консоль контрольной работы № 1 запускается как раньше. API запускается отдельно, секрет задаётся снаружи:

```
export JWT_SECRET="framespace-dev-secret-key-32chars"
java -cp target/kinostudio-crm.jar ru.mirea.kinostudio.api.ApiServer
```